

Constraint Audit — Restructured Analysis

Source: `constraint_audit.md` **Purpose:** Reorganize the original findings around root cause and actual exploitability, rather than treating each finding as an isolated ticket.

Key correction to the original severity ranking

The original audit sorts by severity label. Actual priority should weight *exploitability* × *blast radius*, which changes one classification and one ranking:

- **H-5 (`deleteLink` ownership check) is mislabeled.** It's filed under "HIGH — Foreign Key Violations," but it isn't a crash risk — it's a **cross-tenant IDOR (insecure direct object reference)**. The project ownership check passes, but the delete itself runs on `linkId` alone with no scoping to that project. Any authenticated user in any org can delete any link in the system by guessing or enumerating IDs, regardless of which business owns it. This should be **priority 1**, not priority 7 — it's a live authorization bug affecting live customer data, not a theoretical race condition, and the fix is nearly free since the correct composite-key pattern already exists next to it in `unlinkAssetFromProject`.
 - **C-1/C-2/C-3 deserve top billing, but for a sharper reason than "race condition."** All three converge on `submitIntakeForm`, which the original audit itself notes is public and unauthenticated. That means the exposure isn't "two tabs open" — it's reachable by bots, retries, or a client double-submitting on a flaky connection, with no auth friction slowing concurrent hits. Treat this as the adversarial entry point it is, not a UX edge case.
-

Findings grouped by root cause

Most of the 17 original findings collapse into five underlying patterns. Fixing the pattern once resolves multiple line items at once.

1. Read-then-write ID generation (TOCTOU)

Covers: C-1, C-2

The fix isn't "add a transaction" — it's eliminating the `count()`-derived ID entirely. `invoiceSequence` already demonstrates the correct pattern: a dedicated integer counter column on `Business`, atomically incremented as part of the same write. Add `clientSequence` and `projectSequence` alongside it, and retire the count-based logic everywhere it's duplicated (it appears independently in both `clients/actions.ts` / `projects/actions.ts` and again in `prodp/actions.ts` — worth deduplicating in the same pass).

2. Multi-step writes with no atomic boundary

Covers: C-3, and the same root cause reappears in L-1 (`applyWorkflowPreset` / `restoreDefaults` running `updateMany` then a bulk delete as separate steps)

Once the sequence-counter fix above lands, wrapping `submitIntakeForm` in a single transaction becomes straightforward: client creation, sequence increment, project creation, notes, links, and notification broadcast either all commit or none do. Apply the same transaction boundary to the workflow-stage bulk operations in L-1, so a partially applied `updateMany` can't leave projects referencing a stage that's about to be deleted.

Stack-specific caveat: interactive transactions through Prisma Accelerate have tighter timeout/duration limits than a direct connection. Keep these transactions to pure DB writes — don't include the notification broadcast or other external calls inside the transaction boundary, or a race condition risks being traded for a transaction-timeout failure under load.

3. Missing idempotency on create paths that can legitimately double-fire

Covers: H-3, M-1, M-2

Double-click, a retried network request, or (for M-2) a genuine race between two simultaneous submissions of the same request — the common thread is that an insert can legitimately happen twice for one logical action. The fix is `upsert` on the unique key in place of `create`, applied consistently across all three files as one pass rather than three separate fixes.

4. Missing pre-validation on update paths

Covers: H-1, H-2

Different from #3 — these are updates, so `upsert` doesn't apply. Port the check-then-update pattern `createClient` already uses. Note this alone doesn't fully close the TOCTOU gap (two simultaneous updates can still race past the check), so pair it with catching `P2002` as a fallback rather than treating the pre-check as sufficient on its own.

5. Trusting external event ordering

Covers: H-4

Clerk doesn't guarantee webhook delivery order. The membership handler should defensively upsert the parent `Business`/`User` rows itself before creating the membership — the same approach `broadcastNotification` already uses. Since Upstash QStash is already in the stack, an alternative worth considering is catching the FK failure and re-enqueueing the event for retry after a short delay, which also protects against any *other* undiscovered ordering assumption in the webhook handlers, not just this one.

Items worth reclassifying rather than fixing as-is

Item	Original framing	Reclassification
M-3 — token collision retry	Medium risk	At ~1-in-2 ¹⁹² , this isn't a real risk. The retry-on-(P2002) wrapper is worth building, but as a reusable utility for hygiene — not because this specific collision is a meaningful threat.
M-4 — invoice prefix reuse	Conceptual gap	This is a business-logic decision that hasn't been made explicit, not a DB integrity gap. Either disallow reusing a previously-used prefix, or denormalize the prefix onto each invoice at creation time so old numbers stay valid independent of later settings changes. Pick one and document it.
L-2/L-3/L-4 — soft/no FK fields	Grouped as one integrity gap	These need different answers, not one fix. (AuditLog) surviving user/entity deletion is very likely the <i>correct</i> design for an audit trail — document it as intentional so it doesn't get "fixed" into a compliance problem later. (TimeEntry.userId) orphaning silently is more likely an actual bug, and probably warrants a real relation with (onDelete: SetNull).
L-5 — payment overpayment	Data integrity gap	Add a DB-level check constraint ((amountPaidCents <= totalCents)) in addition to the app-level check. An app-only guard is exactly the kind of rule that gets silently bypassed by the next write path someone adds without knowing it exists.

Revised priority order

Priority	Item(s)	Why it moves here
1	H-5	Live cross-tenant authorization bug, not a crash risk. Cheap fix, highest real-world exposure.
2	C-1 / C-2 / C-3	Public, unauthenticated entry point (<code>submitIntakeForm</code>); one combined fix (sequence counters + transaction wrapper) resolves all three.
3	H-4	Webhook ordering failures are silent and can quietly corrupt onboarding state.
4	M-2	Also reachable via a public-facing link (feedback response), same TOCTOU shape as C-1/C-2.
5	H-1 / H-2 / H-3	Low-effort, high-frequency crash paths (normal edit/link flows).
6	M-1	Same idempotency fix as H-3, bundle together.
7	L-1	Bundle with the C-3 transaction work — same root cause.
8	L-5, M-4	Business-logic guards, not crash risks — real but lower urgency.
9	M-3, L-2/L-3/L-4	Hygiene / documentation, not active bugs.

Gaps not covered by the original audit

1. **Webhook idempotency beyond ordering** — the audit checks upsert-safety for *ordering*, but Clerk can also redeliver the same event. Worth confirming there's dedup for the case where `organizationMembership.created` fires twice for the same event ID.
2. **Rate limiting on public endpoints** — fixing the DB race on `submitIntakeForm` and the feedback-response flow stops the *crash*, but both remain unauthenticated endpoints that could be scripted to inflate sequence counters or spam records. That's an infra/product concern adjacent to this audit, not inside it, but it shares the exact same attack surface as C-1/C-3.